From Zero to Vue

A Four-Week Compounding Challenge

JavaScript → TypeScript → Vue Basics → Vue Advanced

Target pace: ~40 hours / week

How to Use This Plan

Each week is a formal challenge. Treat the acceptance criteria as a contract: the week is not done until every box is checked. The apps escalate on purpose. You are not rebuilding the same To-do list four times. Each project reuses the muscles from the previous week (persistence, async, typing) while forcing genuinely new patterns, so knowledge compounds instead of plateauing.

- Read the full challenge before writing any code. Sketch the data model first.
- Meet the acceptance criteria before touching the stretch goals.
- Stretch goals are where the real learning lives. Do not skip them if you finish early.
- Commit to git daily. By week 4 your history should tell the story of how you built it.

Week 1 — JavaScript Fundamentals

> **THE CHALLENGE**
> Build a fully functional To-do list application using only HTML, CSS, and vanilla JavaScript. No frameworks, no libraries, no build tools. Just the language and the browser.

Why this challenge

The To-do list is the canonical starting project because it exercises every core browser skill at once: rendering data to the DOM, reacting to user events, mutating state, and persisting it. Doing it without a framework forces you to understand what frameworks actually do for you before you lean on one.

Topics to cover

- DOM: selecting, creating, appending, and removing elements; event delegation.
- Events: click, input, submit, keyboard events; preventDefault and event bubbling.
- Array methods: map, filter, reduce, find, some, every, sort.
- ES6+ syntax: arrow functions, destructuring, spread/rest, template literals, modules (import/export).
- Closures and scope; the difference between let, const, and var.
- Persistence with localStorage; JSON.parse and JSON.stringify.
- Async foundations: promises, async/await, and a simulated API using setTimeout.

Acceptance criteria

The week is complete when all of the following are true:

- ☐ A user can add a task via an input field and a button (and by pressing Enter).
- ☐ Each task can be marked complete/incomplete, with a clear visual distinction.
- ☐ Each task can be deleted.
- ☐ Each task can be edited in place (double-click or an edit control).
- ☐ Tasks can be filtered by All, Active, and Completed.
- ☐ A live counter shows how many active tasks remain.
- ☐ All tasks persist across a full page reload via localStorage.
- ☐ Empty or whitespace-only tasks are rejected.
- ☐ Code is split into ES modules, not one giant file.

Stretch goals

- Drag-and-drop reordering of tasks (native HTML5 drag events).
- A simulated backend: wrap load/save in async functions with an artificial setTimeout delay, and show a loading state. This pre-loads the async mindset you will need in week 2 and beyond.
- A "Clear completed" action and a bulk "Mark all complete" toggle.
- Keyboard-only usability: full task lifecycle without touching the mouse.

A Note on Pacing

Forty hours a week is plenty of time, but the trap is comfort. The temptation each week is to polish the familiar rather than attempt the hard, unfamiliar stretch goals. Resist it. The acceptance criteria guarantee a working app; the stretch goals are what turn four working apps into a front-end engineer. If a week feels easy by Wednesday, you are not reaching far enough into the stretch list.

Build in public if you can. Push each project to GitHub, write a short README explaining the decisions you made, and by the end of the month you will have four portfolio pieces that visibly escalate in sophistication.